

企业知识工作台产品设计文档

面向知识复用与决策准备的 AI workflows 设计

版本：对外展示版 v1.1

用途：作为作品集模块 1「企业知识工作台」可挂载 PDF 的内容母稿。

定位：产品级原型 / 可演示 MVP，用于验证企业知识复用、可信上下文层与 Agent workflows 的产品化路径。

边界：本文档不声称该系统已上线企业生产环境，也不包含敏感资料、真实企业内部文档或私人文件路径。

0. 文档说明

本文档用于展示「企业知识工作台」这一 AI

产品责任单元的设计思路。它不是技术实现说明书，也不是完整工程文档，而是围绕一个企业级 AI 应用场景，说明如何从业务问题出发，判断 AI 介入点，定义产品形态，并通过可信上下文、Agent workflows、人机协同和评估机制形成可维护的产品化闭环。

本文档重点回答以下问题：

1. 企业知识复用与决策准备场景中，真实业务问题是什么？
2. AI 应该在哪些工作节点介入？
3. 基础模型能力增强后，为什么仍然需要可信上下文层？
4. RAG、Agent、Memory、Human Review、badcase 和 eval 分别承担什么产品作用？
5. 如何验证这个工作台不是一次性 demo，而是可迭代、可追踪、可回滚的产品级原型？

本文档中的案例来自一个产品级原型。原型阶段曾使用本地资料、项目文档和脱敏样例进行验证，用于模拟企业知识复用与项目复盘场景。对外展示时，本文档不暴露私人资料、真实文件路径或不可公开业务信息。

1. 项目概述

1.1 项目背景

在企业环境中，大量知识资产并不只存在于正式知识库中，而是分散在项目文档、会议纪要、方案评审材料、复盘报告、业务规则、历史案例和跨部门沟通记录中。随着资料持续增加，传统目录、关键词搜索和人工经验越来越难支撑真实工作：

- 项目成员记得“以前讨论过类似问题”，但不记得具体文件；
- 新接手成员需要花大量时间了解项目背景和历史判断；
- 管理者或项目负责人需要快速准备决策依据，但资料散落在不同位置；
- 项目复盘写完后没有进入可检索、可复用、可迭代的组织记忆；
- 同类问题反复调研、反复沟通、反复总结。

因此，本项目关注的不是“做一个知识库问答机器人”，而是：

面向企业知识分散、历史经验难复用和决策依据难追溯的问题，将资料检索、依据追溯、项目复盘、经验沉淀和评估迭代组织为一个可验证、可追溯、可维护的 AI 知识工作台。

1.2 产品定义

企业知识工作台 是一个面向知识复用与决策准备场景的 AI 工作台模块。它负责将分散资料转化为可检索、可追溯、可复盘、可沉淀、可迭代的知识 workflow。

它的核心价值不是替用户“直接生成答案”，而是帮助用户完成一段知识工作：

找到资料 → 追溯依据 → 整理结论 → 形成复盘 → 提炼后续判断参考 → 沉淀经验 → 通过 badcase 和 eval 持续迭代。

1.3 当前项目状态

当前项目是产品级原型 / 可演示 MVP，重点验证以下能力：

- 面向指定资料范围的语义检索；
- 基于来源的可追溯问答；
- 从单轮问答升级到 Agent 工作流；
- 使用 Router、Tool、Memory、Human Review 组织连续知识任务；
- 通过 badcase、eval、changelog 和 rollback 支持持续迭代；
- 明确边界：不自动写入、不替代人工判断、不包装成已上线生产系统。

2. 业务背景与问题定义

2.1 企业知识的典型来源

企业知识通常来自多个系统和材料类型：

- 项目文档与阶段性方案；
- 会议纪要与决策记录；
- 项目复盘与经验总结；
- 业务规则、流程规范和操作手册；
- 客户反馈、售后记录和质量问题记录；
- 内部沟通记录、评审意见和跨部门协作材料；
- 历史案例、竞品分析和管理汇报材料。

这些知识往往不是缺少，而是缺少一个能够持续复用的产品化机制。

2.2 核心业务问题

问题	具体表现	影响
资料分散	项目资料、会议记录、复盘文档和规则说明散落在不同位置	用户需要依赖记忆、关键词和人工询问
经验难复用	历史项目经验留在个人经验或零散文档中	新项目重复踩坑，组织经验无法沉淀
决策依据难追溯	结论被复用时，来源、依据和历史判断难以确认	影响决策可信度和责任边界
接手成本高	新成员需要反复问人、翻文件、补背景	协作效率下降，项目连续性弱
复盘难形成资产	复盘材料写完后没有进入持续检索和迭代机制	复盘停留在文档层，没有成为组织记忆

2.3 为什么传统搜索不够

传统搜索解决的是“找到文件”，但企业知识工作台要解决的是“基于资料完成一段知识工作”。

方式	能解决什么	局限
文件夹目录	适合结构清楚、文件较少的场景	文件变多后依赖记忆，跨目录查找困难
关键词搜索	能查标题、术语和精确文本	对模糊意图、同义表达、跨文档整合支持弱
全文搜索工具	能快速定位包含关键词的片段	不能组织答案，也不能判断来源是否足够
直接调用大模型	适合通用解释、写作润色和低风险生成	不知道企业指定资料，难以稳定追溯依据
人工整理索引	可控、准确	维护成本高，资料更新后容易过期

2.4 为什么 AI 适合介入

企业知识复用场景天然包含大量非结构化文本、历史材料、上下文判断和总结任务。AI 的价值不在于完全替代人，而在于承担以下工作：

- 将自然语言问题转化为可检索意图；
- 从分散文档中召回相关片段；
- 基于来源归纳结论；
- 将历史资料整理为复盘草稿；
- 在多轮任务中保留上下文；
- 通过 badcase 和 eval 将错误转化为下一轮迭代素材。

但企业知识场景不能只追求“回答流畅”，还必须保证来源、边界、更新、核验和责任。因此，本项目不是直接让模型自由生成，而是在模型和企业知识资产之间设计可信上下文层与人机协同机制。

3. 用户角色与核心任务

3.1 用户角色

用户角色	典型诉求
项目负责人 / 产品经理	快速查找历史方案、复盘资料和决策依据，准备项目汇报或方案判断
新接手成员	快速理解项目背景、关键节点、历史判断和相关资料
管理者 / 决策参与者	查看结论依据、历史案例、风险点和复盘结论
PMO / 知识管理角色	维护项目经验、沉淀组织知识、减少重复沟通和重复调研

3.2 用户核心任务

用户表面上是在“查资料”，实际要完成的是一组知识任务：

- 1. 找回相关资料；
- 2. 找到结论依据；
- 3. 跨文档整理结论；
- 4. 形成项目复盘；
- 5. 准备决策或汇报材料；
- 6. 提炼后续判断参考；
- 7. 沉淀经验；
- 8. 基于上一轮结果连续追问。

3.3 用户任务分层

层级	用户诉求	产品能力
找得到	不记得文件名也能找回资料	语义检索、来源召回
信得过	答案要有依据，不能凭空生成	引用溯源、原文核验
用得顺	不想每轮重复交代背景	Memory、连续追问
能复盘	历史经验能变成结构化材料	复盘模板、经验提炼
可维护	资料更新后系统要同步	索引更新、版本记录
可治理	输出不能黑盒生成	人审、权限、日志、评估

4. 当前流程与 AI 介入点

4.1 传统知识使用流程

传统知识使用流程与痛点



图 1：线框流程图

传统流程的问题不是完全找不到资料，而是资料无法稳定转化为可用结论、复盘经验和组织记忆。主要痛点包括：

流程节点	典型痛点
人工查找	依赖关键词、目录结构和个人记忆
跨文档阅读	阅读成本高，容易遗漏关键上下文

流程节点	典型痛点
手动整理结论	依据与结论容易分离，后续难以核验
经验再次分散	复盘结果难以进入可持续复用的组织记忆

4.2 AI 介入点表

工作节点	传统方式的问题	AI 介入点	产品价值
找资料	依赖关键词搜索，容易漏资料	语义检索 + 相关片段召回	降低查找成本
找依据	搜到文件后仍需人工翻阅	引用溯源 + 证据片段提取	让结论可核验
整理结论	需要人工跨文档总结	多文档归纳 + 结构化总结	提升整理效率
项目复盘	依赖个人经验和记忆	复盘模板 + 关键经验提炼	沉淀组织记忆
生成参考建议	搜索结果不能直接转行动	基于资料形成后续判断参考	从问答走向知识工作流
迭代优化	错误输出难复盘	badcase + eval set + 版本记录	支持持续改进

4.3 业务问题 → AI 介入点 → 产品责任单元

业务问题 → AI 介入点 → 产品责任单元

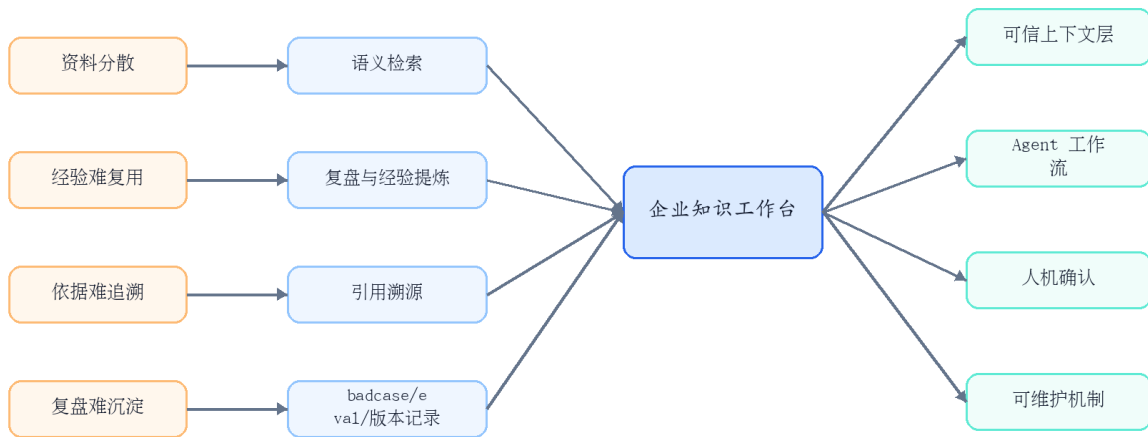


图 2：线框流程图

4.4 人机分工原则

环节	AI 可做	人必须确认
资料召回	召回相关片段、展示来源	判断是否采信
结论整理	生成结构化总结草稿	确认结论是否准确
项目复盘	提炼关键事件、问题和经验	判断复盘结论和改进方向
参考建议	基于资料生成下一步建议	决定是否执行
知识沉淀	生成待沉淀内容	确认后写入或发布

AI 输出在本项目中被定义为“带来源的工作草稿和决策辅助”，不是自动决策结果。

5. 产品责任单元定义

5.1 产品责任单元

企业知识工作台的产品责任单元，是围绕“知识复用与决策准备”这一场景，将资料检索、依据追溯、项目复盘、经验沉淀和评估迭代组织为一个可维护的 AI workflow 模块。

它不是一个孤立问答机器人，也不是一个简单文档总结工具，而是位于用户知识任务和企业知识资产之间的产品化承接层。

5.2 产品能力模块

能力模块	产品作用
知识检索	找到相关资料和依据
引用溯源	让关键结论可回到原文核验
知识任务型问答	从“找答案”升级为“完成知识整理任务”
项目复盘	把历史资料转成结构化复盘
Memory 机制	保留当前任务上下文、历史偏好和当前目标
Human Review	在关键输出前保留人工判断
badcase 复盘	将错误案例转化为迭代素材
Eval Set	用固定问题集做版本回归
版本管理	记录 prompt、workflow、数据索引和输出机制的变化

5.3 产品边界

当前产品责任单元不做以下事情：

- 不做完整企业权限系统；
- 不做完整任务管理系统；
- 不自动写入、删除或发布企业资料；
- 不替代人工决策；
- 不声称已上线生产环境；
- 不替代模块 2 的需求、问题、风险和任务流转机制。

6. 关键产品判断：基础模型增强后，为什么仍需要可信上下文层？

6.1 不再把 RAG 当主角

基础模型能力增强后，很多通用问答、简单文本生成和低风险信息整理场景，确实可以直接依赖模型完成。因此，本项目并不把 RAG 作为最终产品主角，也不把“用了 RAG”当成核心卖点。

更准确的判断是：

RAG 作为一种实现方式，承担的是可信上下文层功能；未来形态可能变化，但企业知识场景仍然需要来源、边界、更新和核验机制。

6.2 企业知识场景为什么需要可信上下文层

在企业知识复用与决策准备场景中，核心问题不是模型能不能生成答案，而是：

- 答案是否来自指定资料；
- 是否能追溯到原文依据；
- 是否能随着企业知识更新而更新；
- 是否符合资料范围和权限边界；
- 是否能被人工复核；
- 是否能沉淀为组织记忆；
- 是否能通过 badcase、eval 和版本机制持续改进。

因此，本项目中需要在基础模型和企业知识资产之间设计一层知识与上下文中间层。它不是为了增加架构复杂度，而是为了让模型输出进入企业知识场景时有来源、有边界、有依据、可核验、可评估、可回滚。

6.3 对比表

问题	直接调用基础模型	可信上下文层
企业内部资料	不保证覆盖最新内部资料	接入指定资料范围
引用与依据	难以稳定追溯	可回到原文核验
知识更新	依赖模型更新或手动上下文输入	可随资料库持续更新
权限边界	难以按资料范围管理	可按资料范围和权限设计
可核验性	输出可能合理但难验证	输出可回到原文核验
组织记忆	难以沉淀为企业资产	可结合复盘、badcase 和版本记录沉淀

6.4 结论

基础模型越强，越需要产品经理判断哪些场景可以轻量直连模型，哪些场景必须通过可信上下文层来保证知识边界、引用依据和组织可维护性。

本项目的产品判断是：

在企业知识复用与决策准备场景中，RAG 不应被表达为“过时或先进”的技术选择，而应被重新定位为知识与上下文中间层的一种实现方式。真正的产品价值在于，在可信上下文之上构建 Agent workflow、人机确认、评估机制和可维护迭代机制。

7. 方案选择与产品路线判断

7.1 方案对比

方案	适合解决	不适合本项目的原因	结论
直接 LLM	通用问答、写作润色、低风险生成	不知道指定企业资料，缺少稳定来源	可作为生成层，不作为知识来源层
关键词搜索	精确查找文件名、标题、术语	不支持模糊意图、跨文档总结和任务型输入	保留为辅助
Fine-tuning	固定格式、风格、分类稳定性	不适合频繁更新知识和引用追溯	暂不优先
RAG / 可信上下文层	接入资料、引用溯源、可更新	需要处理数据、chunk、索引和检索质量	V1 主方案
Agent 工作流	路由、工具调用、连续任务、人机确认	引入 Router、Memory 和 Tool 调用风险	V2 升级方向

7.2 企业知识场景的 AI 方案选择判断树

企业知识场景的 AI 方案选择判断树

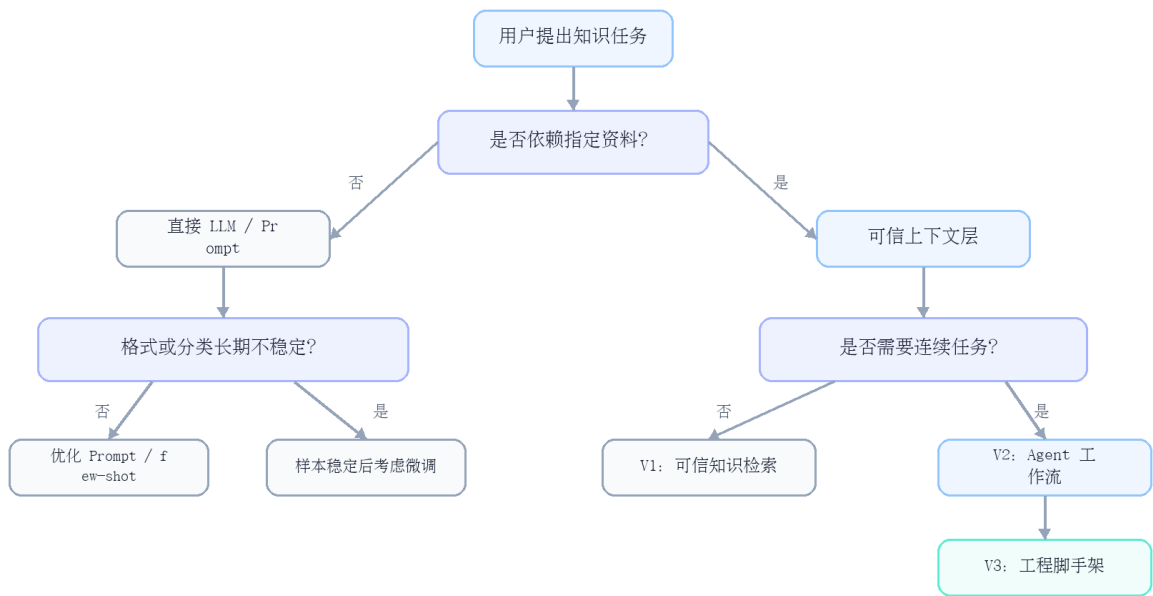


图 3：线框流程图

7.3 关键判断

- RAG 解决“模型不知道指定资料、答案需要来源”的问题；
- Agent 解决“系统要判断路径、调用工具、支持上下文”的问题；
- Fine-tuning 更适合解决“怎么答、怎么分类、怎么保持格式”的问题；
- 工程脚手架解决“如何让 AI 工作流可维护、可评估、可回滚”的问题。

8. 产品演进路径：V1 / V2 / V3

8.1 演进主线

本项目的演进不是“技术越来越复杂”，而是企业知识 workflow 从可信上下文，到任务执行，再到可维护机制的产品化演进。

阶段	核心问题	产品判断
V1: 可信知识检索	为什么需要可信上下文层？	先解决资料找得到、答案有依据、结论可复核
V2: Agent 工作流	为什么 RAG 不够？	用户要完成连续知识任务，需要 Router、Tool、Memory 和 Human Review
V3: 可维护 MVP	为什么 Agent 还不够？	没有 eval、badcase、changelog、rollback, AI 工作流仍然只是一次性 demo

8.2 V1 / V2 / V3 产品化演进图

V1 / V2 / V3 产品化演进



图 4：线框流程图

8.3 V1：为什么需要可信上下文层？

在企业知识场景中，问题不是模型是否知道通用答案，而是答案是否基于指定资料、是否可引用、是否可核验、是否可更新。因此，V1 的目标是先建立可信检索与引用溯源链路。

8.4 V2：为什么 RAG 不够？

用户真正要完成的不是“问一个问题”，而是一段连续知识工作：查资料、找依据、整理结论、形成复盘、提炼后续判断参考。因此，V2 需要从固定问答链路升级为 Agent 工作流。

8.5 V3：为什么 Agent 工作流还不够？

如果没有版本、评估、badcase、memory policy 和 rollback, Agent 工作流仍然只是一次性 demo，无法支撑持续迭代和团队级维护。因此，V3 需要工程脚手架和评估机制。

9. V1 设计：可信知识检索与可追溯问答

9.1 V1 目标

V1 的目标不是做复杂 Agent，而是先验证最基础也最关键的闭环：

资料找得到、答案有依据、用户能回到来源复核。

9.2 V1 MVP 范围

能力	说明
资料接入	接入指定资料范围，支持原型阶段文档验证
文本清洗与切分	将资料转成可检索片段
语义检索	支持自然语言问题召回相关资料
来源展示	展示来源文件、标题、片段和相关性
基于来源回答	用模型组织答案，但约束其基于来源回答
不确定性提示	资料不足时明确说明，而不是强行生成
索引刷新	支持资料更新后的重新索引和状态提示

9.3 V1 数据流

V1 可信知识检索流程

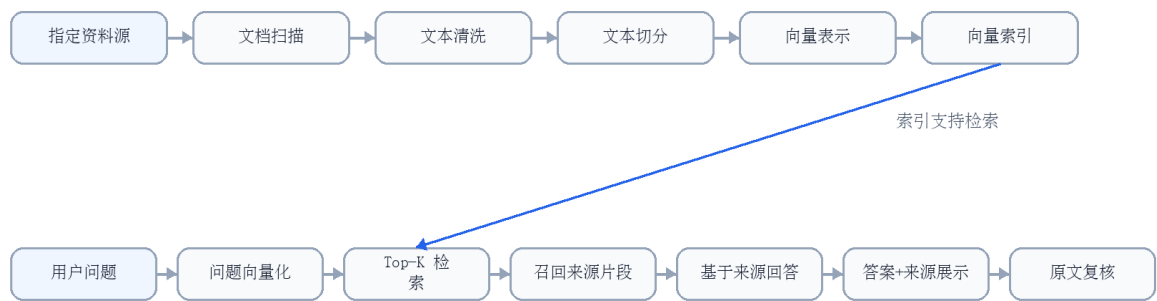


图 5：线框流程图

9.4 关键设计点

设计点	产品意义
Chunk 策略	影响资料是否能被正确召回，不是纯技术细节
元数据	帮助区分项目、来源、时间和资料范围
来源展示	提升可信度，让用户能回到原文核验
不确定性提示	避免模型在证据不足时编造答案
索引状态	让用户知道知识库是否更新，避免旧资料污染

10. V2 设计：Agent workflows与任务型知识助手

10.1 V2 升级目标

V2 的目标是把 V1 固定的“检索 → 回答”链路，升级为可判断、可调度、可解释的 Agent workflow。

10.2 核心模块

组件	产品意义
路由判断 (Router)	判断用户是在查资料、做复盘、生成总结，还是需要澄清
工具调用 (Tool)	将检索、总结、复盘等能力封装为可调用的工具
可信上下文层	提供可追溯的知识依据
记忆机制 (Memory)	保留当前项目背景和最近上下文
执行核心 (Agent Core)	组织多步骤执行链路
执行过程展示 (Execution Trace)	展示系统为什么检索、调用了什么工具、用了哪些来源
人工确认 (Human Review)	在关键输出前保留人工判断
badcase 复盘	将错误案例沉淀为下一轮迭代素材

10.3 V2 Agent workflow架构图

V2 Agent workflow架构

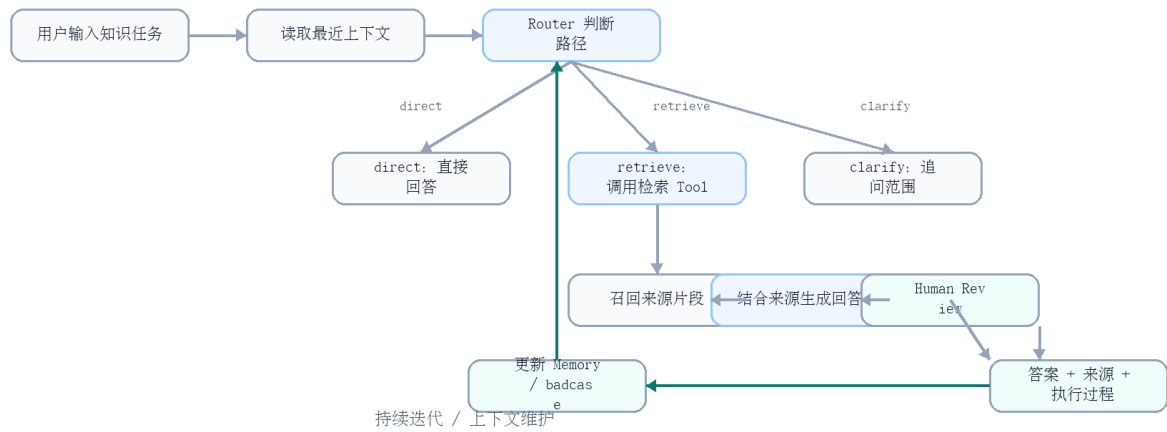


图 6：线框流程图

10.4 只读边界与人工确认

Agent 可以更主动，但不能越权。当前版本默认只读，所有写入、删除、同步、发布类动作都必须人工确认。

动作	当前策略
检索资料	可自动执行
总结资料	可自动生成草稿

动作	当前策略
形成复盘	可生成结构化草稿
写回知识库	需要人工确认
删除或覆盖资料	不自动执行
对外发布	必须人工确认

11. V3 设计：工程脚手架与可维护机制

11.1 V3 目标

V3 的目标是让 AI 工作流不是一次性 demo，而是可迭代、可追踪、可回滚的产品级原型。

工程脚手架不是为了炫技，而是为了回答一个更产品化的问题：

如果这个知识工作台不是演示一次，而是要持续服务一个团队，如何保证它可追踪、可评估、可回滚？

11.2 工程脚手架机制

机制	产品作用
Memory Policy	定义哪些信息可进入记忆，哪些只保留在会话中
Prompt / Workflow Versioning	管理提示词和工作流版本
Changelog	记录每次重要变更
Decision Log	记录关键产品判断和取舍
Badcase Log	收集错误案例并分类复盘
Eval Set	固定测试问题集，比较不同版本效果
Rollback Note	记录失败版本和回退路径
Human Review Checkpoints	明确哪些输出必须人工确认

11.3 产品判断

AI 应用如果没有评估集、badcase、版本记录和回滚路径，就很难从演示原型走向可持续维护的产品能力。V3 的重点不是增加功能，而是建立持续迭代和质量控制机制。

12. 验证体系、badcase 与迭代机制

12.1 验证框架

维度	验证重点
检索层	正确来源是否进入召回结果
生成层	回答是否基于来源，是否承认不确定
Agent 层	Router 是否选对路径，Memory 是否串扰
治理层	badcase 是否进入回归测试，版本是否可追踪

12.2 指标设计

指标	说明
Top-K 命中率	正确来源是否进入召回结果
引用支撑率	引用是否真正支撑结论
Route 命中率	Router 是否选对 direct / retrieve / clarify
Tool 成功率	检索工具是否返回可用来源
Memory 指代准确率	连续追问是否接上上下文
Memory 污染率	主题切换时是否串扰
Trace 完整率	是否展示 route、tool、sources
安全拦截率	写入 / 删除类请求是否要求确认

12.3 代表 Badcase 1：语义相近导致误召回

项目	内容
问题现象	业务用户询问某一历史项目的复盘材料时，系统召回了另一个语义相近但业务对象不同的项目资料
定位过程	两个项目都包含相似高频词，向量相似度高，但业务对象不同
优化动作	增加项目名、source_type、path 等元数据；在 query rewrite 中补充业务对象；增加来源筛选
产品判断	相似度高不等于业务相关，知识工作台需要结合元数据和业务上下文判断相关性

12.4 代表 Badcase 2：来源不足但模型强行回答

项目	内容
问题现象	检索结果不足时，模型仍生成了看似完整的结论
定位过程	这是生成层约束问题，不是资料本身一定缺失
优化动作	prompt 增加“资料中未找到明确依据时必须说明”；输出区分“来源明确支持”和“需要进一步确认”
产品判断	对知识助手来说，可信度优先于流畅度，宁可保守也不要编造

12.5 代表 Badcase 3：Router 误判导致未检索指定资料

项目	内容
问题现象	业务用户提出依赖历史资料的问题，但 Router 误判为 direct，导致回答泛泛
定位过程	问题表面像表达任务，实际依赖指定资料，根因在 Agent 控制层
优化动作	增加规则：出现“指定项目、历史资料、历史案例、复盘材料”等指向词时，优先 retrieve 或 clarify
产品判断	Agent 的第一步不是执行，而是正确判断任务路径；Router 错了，后面的 Tool 再强也发挥不出来

12.6 Badcase 闭环图

Badcase → 定位 → 修复 → 回归测试闭环



图 7：线框流程图

13. 人机协同、安全边界与企业化扩展

13.1 人机协同原则

- AI 输出作为草稿、建议和依据整理；
- 人负责确认、取舍、发布和最终决策；
- 关键结论必须可追溯；
- 高风险动作必须人工确认；
- 系统需要保留执行过程、来源依据和版本记录。

13.2 安全边界

场景	AI 可做	人必须确认
知识检索	召回资料和证据片段	判断是否采信
项目复盘	生成复盘初稿	确认结论和经验
参考建议	给出后续判断参考	决定是否执行
知识沉淀	生成待写入内容	人工确认后写入
对外输出	生成初稿	人工审核后发布

13.3 企业化扩展方向

如需从产品级原型走向企业化应用，需要进一步补齐：

- 权限分层；
- 多用户与空间隔离；
- 企业文档系统连接；
- 审计日志；
- 敏感信息处理；
- 评估面板；
- 版本管理和回滚机制；
- 管理员维护机制。

14. 项目边界与模块 2 边界

14.1 当前项目边界

当前项目是产品级原型 / 可演示 MVP，重点验证企业知识复用与决策准备场景的 AI 产品化路径。它不声称已经具备完整企业生产系统能力。

当前不做：

- 完整权限系统；
- 完整多用户协作；
- 完整任务管理系统；
- 自动写入 / 删除 / 发布；
- 替代人工判断；
- 替代模块 2 的业务需求流转。

14.2 与模块 2 的边界

模块	核心对象	回答的问题
模块 1：企业知识工作台	知识资产、历史经验、决策依据	企业过去积累的资料、经验和判断，如何被检索、引用、复盘和沉淀
模块 2：AI 业务需求流转工作台	当前需求、问题、风险、责任链路、状态流转	新业务信号产生后，如何被识别、确认、分流、追踪和复盘

因此，模块 1 可以涉及“复盘建议 / 后续判断参考”，但不展开完整任务状态、责任人、优先级、周报和推进闭环。这些内容留给模块 2。

14.3 与模块 4 的关系

模块 4 负责将本项目中的评估、治理、badcase、人审、版本管理和中间层必要性进一步抽象为跨场景方法论。模块 1 是一个具体知识场景案例，模块 4 是评估治理框架。

附录 A：可对外展示的 Mermaid 图表清单

1. 传统知识使用流程与痛点图；
2. 业务问题 → AI 介入点 → 产品责任单元图；
3. 企业知识场景的 AI 方案选择判断树；
4. V1 / V2 / V3 产品化演进图；
5. V1 可信知识检索流程图；
6. V2 Agent 工作流架构图；
7. Badcase → 定位 → 修复 → 回归测试闭环图。

附录 B：对外展示时不展开的内容

这些内容保留在内部母版中，用于面试准备、后续页面改造和工程说明，不作为对外 PDF 主体展示：

- 详细 PLAN.md 全文；
 - 具体代码接口；
 - 私人文件路径；
 - 过细的 chunk / embedding / vector database 参数；
 - 全量 badcase；
 - 面试话术；
 - 内部训练清单；
 - 未脱敏的真实资料样例。
-

结语

企业知识工作台的核心，不是证明一个 RAG demo 能运行，而是展示一个企业知识复用与决策准备场景如何被拆解成 AI 产品责任单元。

它的产品化路径可以概括为：

先通过可信上下文层解决“资料找得到、答案有依据、结论可复核”；再通过 Agent workflow 解决“用户要完成连续知识任务”的问题；最后通过 badcase、eval、版本管理和人审机制，让 AI workflow 从一次性 demo 走向可维护、可评估、可回滚的产品级原型。